SQL + Next.js
6-Week Weekend Learning Plan

Build a Student Management System while learning SQL from CRUD to advanced concepts.

Duration	6 weekends (5 hours each) — 30 hours total
Database	PostgreSQL (local install or Neon cloud free tier)
Framework	Next.js 14+ (App Router) + TypeScript + Tailwind CSS
DB layer	Raw SQL via `pg` library (weeks 1–5), Prisma ORM (week 6)
Project	Student Management System
Outcome	Production-style CRUD app + solid SQL fundamentals

Overview & Goals

This plan turns 30 hours of weekend study into a working, deployable Student Management System. You will write real SQL by hand for the first five weekends so the concepts stick, then graduate to Prisma ORM in week 6 to feel the productivity difference once you understand what it abstracts away.

What you will know by week 6

- Design normalised relational schemas (1:1, 1:N, M:N relationships).
- Write CRUD, JOINs, aggregations, subqueries, CTEs, and window functions confidently.
- Read query plans (EXPLAIN ANALYZE) and add indexes where they matter.
- Build a Next.js App-Router CRUD app with Server Actions and forms.
- Wrap raw SQL behind safe parameterised queries (no SQL injection).
- Migrate a raw-SQL codebase to Prisma and understand the trade-offs.
- Deploy a full-stack app to Vercel with a managed Postgres database.

How to use this plan

- Each week has roughly five 1-hour blocks; treat them as a guide, not a stopwatch.
- Type every query yourself — do not copy-paste. Muscle memory is the point.
- End each weekend by committing your work to Git with a descriptive message.
- If a topic feels shaky, stay on it. The plan compounds — week 5 needs week 3.
- Keep a tiny notes file (notes.md) with `gotchas` you hit; review it weekly.

Project domain — Student Management System

You will progressively build these tables, one or two per week:

Table	Columns (highlights)	Introduced
students	id, name, email (unique), dob, enrollment_date	Week 1
courses	id, code (unique), title, credits	Week 3
enrollments	student_id, course_id, semester (M:N bridge)	Week 3
grades	enrollment_id, grade, graded_at	Week 4
users	id, email, password_hash, role (for auth)	Week 6

Week 1 — Foundations — Postgres, schemas, SELECT, and Next.js skeleton

Get a clean environment, create your first table, learn how a Next.js App Router project is wired up, and read your first rows from Postgres in a Next.js page. By Sunday night you have a running app that lists students from a real database.

Hourly breakdown (5 hours)

Hour	Focus	Concrete output
1	Install PostgreSQL locally OR sign up for a free Neon project. Install pgAdmin or DBeaver as a GUI.	Postgres reachable, connection string saved.
2	SQL basics: CREATE DATABASE, CREATE TABLE, data types (INT, TEXT, VARCHAR, DATE, TIMESTAMPTZ, BOOL	`students` table created with proper constraints.
3	INSERT (single and multi-row), SELECT, WHERE, AND/OR, BETWEEN, IN, LIKE, IS NULL, ORDER BY, LIMIT/OFFSET	20 seed rows inserted, 8 SELECT queries written.
4	Create Next.js project: `npx create-next-app@latest --typescript --tailwind --app`. Install `pg` and `dotenv`. Add `.env.	Next.js dev server running on localhost:3000.
5	Write `lib/db.ts` that exports a singleton `pg.Pool`. Build `/students` page (Server Component) that queries and renders all	Page shows real DB data.

SQL concepts introduced

- Data types and choosing the right one (TEXT vs VARCHAR, TIMESTAMPTZ vs TIMESTAMP).
- Constraints: PRIMARY KEY, UNIQUE, NOT NULL, CHECK, DEFAULT.
- Auto-increment IDs: SERIAL vs GENERATED ALWAYS AS IDENTITY (prefer the latter).
- Basic SELECT with WHERE, ORDER BY, LIMIT, OFFSET.
- Pattern matching with LIKE / ILIKE.

Next.js concepts introduced

- App Router file conventions: page.tsx, layout.tsx, loading.tsx.
- Server Components vs Client Components — the default and when to opt out.
- Reading environment variables on the server only.
- Using `pg` Pool as a module-scope singleton (avoid creating a pool per request).

Minimal lib/db.ts (singleton pool)

```
import { Pool } from 'pg';
declare global { var pgPool: Pool | undefined; }
export const pool = global.pgPool ?? new Pool({
  connectionString: process.env.DATABASE_URL,
});
if (process.env.NODE_ENV !== 'production') global.pgPool = pool;
```

Practice exercises

- Write a query that returns students enrolled in the last 30 days, newest first.
- Write a query that returns students whose email ends with '@accenture.com'.
- Add a CHECK constraint that ensures `dob` is at least 16 years before today.
- Add 5 more rows where email is missing — observe what NOT NULL prevents.

- Make `/students` accept a `?search=` query param that filters by name (use parameterised query, NOT string concat).

Definition of done for this week

- Repo initialised with Git, first commit pushed (optional: to GitHub).
- `students` table exists with seed data.
- Visiting `/students` shows live DB rows in a styled table.
- Zero hard-coded SQL parameters — all values pass through `$1`, `$2` placeholders.

Safety rule for the whole plan: never build SQL by concatenating user input. Always use parameterised queries: `pool.query('SELECT ... WHERE name = $1', [name])`. This is the single most important habit to form.

Week 2 — CRUD end-to-end with raw SQL and Server Actions

Turn the read-only page from week 1 into a fully working CRUD app. You will write INSERT / UPDATE / DELETE by hand and learn the Next.js Server Actions pattern that lets a form call server code without writing a separate API route.

Hourly breakdown (5 hours)

Hour	Focus	Concrete output
1	UPDATE … SET … WHERE, DELETE FROM … WHERE, RETURNING clause for getting back the affected rows.	10 update/delete queries written by hand against `students`.
2	Build `app/students/actions.ts` with `createStudent`, `updateStudent`, `deleteStudent` server actions. Use `revalidatePath	Three server actions complete and work.
3	Build the `<StudentForm>` Client Component that posts to the create action. Form validation on the client + revalidatio	Form used for both create and edit.
4	Add edit page (`/students/[id]/edit`) and dynamic route. Add delete confirmation. Add success/error flash messages.	Full create/read/update/delete works in the browser.
5	Add Zod validation on the server side. Display field-level errors. Polish styling with Tailwind.	Invalid email/missing name shows inline error; DB never receives bad da

SQL concepts introduced

- UPDATE with WHERE — and the danger of forgetting the WHERE.
- DELETE with WHERE; soft delete vs hard delete (briefly).
- RETURNING * — get the new/updated row in a single round trip.
- Conditional UPDATE: only update non-null fields (COALESCE pattern).

Next.js concepts introduced

- Server Actions ('use server') — what runs where.
- `revalidatePath` and `revalidateTag` for cache invalidation.
- Dynamic routes `[id]`.
- Progressive enhancement — forms work even with JS disabled.
- Zod for input validation on the server boundary.

Pattern: server action with parameterised SQL + revalidation

```
'use server';
import { pool } from '@/lib/db';
import { revalidatePath } from 'next/cache';
export async function createStudent(form: FormData) {
  const name = String(form.get('name'));
  const email = String(form.get('email'));
  await pool.query(
    'INSERT INTO students(name, email) VALUES ($1, $2)',
    [name, email]
  );
  revalidatePath('/students');
}
```

Practice exercises

- Add a 'soft delete' column `deleted_at` and change DELETE to UPDATE; hide soft-deleted rows from `/students`.
- Make UPDATE skip fields that the form left blank (COALESCE pattern).
- Add a uniqueness error path: catch Postgres error code `23505` and show 'Email already used'.
- Add server-side rate limit: only allow 5 inserts per minute per IP (any approach is fine).
- Add `created_at` / `updated_at` columns with `DEFAULT now()` and a trigger to update `updated_at` on UPDATE.

Definition of done for this week

- All four CRUD operations work end-to-end.
- Server actions return clearly typed success/error results.
- No SQL injection surface (parameterised everywhere).
- Form re-renders with field-level errors on invalid submit.

Week 3 — Relationships and JOINs — courses and enrollments

Move from a single table to a real relational schema. Add courses and a many-to-many enrollments bridge. This is the weekend you internalise INNER, LEFT, RIGHT, FULL OUTER JOINs by building screens that need each one.

Hourly breakdown (5 hours)

Hour	Focus	Concrete output
1	Design `courses` and `enrollments`. FOREIGN KEY, ON DELETE CASCADE vs RESTRICT vs SET NULL. Composite pr	Two new tables created with proper FKs.
2	INNER JOIN — only matching rows. Multi-table joins. Joining on more than one column.	10 INNER JOIN queries written.
3	LEFT JOIN — keep all from the left side. When NULLs appear. RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN (briefly).	Queries: students with no enrollments, courses with no students.
4	Build courses CRUD (reuse week-2 patterns). Build enrollment page that lets you assign a student to a course for a seme	Courses and enrollment pages working.
5	Build `/students/[id]` detail page showing the student plus all their enrolled courses (single JOIN query, grouped in the UI).	Detail page shows joined data correctly.

SQL concepts introduced

- FOREIGN KEY constraints and ON DELETE/UPDATE actions.
- INNER JOIN, LEFT/RIGHT/FULL OUTER JOIN, CROSS JOIN.
- Self joins and aliasing tables.
- Joining on composite keys.
- NULL semantics in joins (why LEFT JOIN + WHERE x IS NULL is the 'anti-join' pattern).

Next.js concepts introduced

- Composing multiple queries on one page vs one rich JOIN query.
- Shaping query results into nested objects in the data layer (not the JSX).
- Loading UI with `loading.tsx` and Suspense.

Anti-join example — students with no enrollments

```
SELECT s.id, s.name
FROM students s
LEFT JOIN enrollments e ON e.student_id = s.id
WHERE e.student_id IS NULL;
```

Practice exercises

- Write a query that returns each course's title with its enrolled student count, including courses with zero enrolments.
- Write a query that returns pairs of students who share at least one course (self-join on enrollments).
- Add ON DELETE CASCADE to enrollments → students; observe what happens when you delete a student.
- Build a page that shows a course's roster (course title + list of students).
- Write the 'students enrolled in BOTH course X and course Y' query — there are several ways; try at least two.

Definition of done for this week

- Three connected tables with referential integrity.
- Working CRUD for students and courses.
- Enrollment screen that prevents duplicate (student_id, course_id, semester) rows.
- At least one screen powered by a JOIN that touches all three tables.

Week 4 — Aggregations, GROUP BY, subqueries — your first reports

Add a `grades` table and build a small reports/dashboard area. This is the medium-SQL weekend: GROUP BY, HAVING, the aggregate functions, and subqueries (both scalar and correlated). You will also wire up pagination properly.

Hourly breakdown (5 hours)

Hour	Focus	Concrete output
1	Create `grades` table linked to enrollments. Se[illegible]	[illegible] MAX, FILTER (Postgres
2	GROUP BY — what it actually does. HAVING [illegible]	[illegible] hand). GROUPING SETS / ROLLUP / CUBE (b
3	Subqueries: scalar, IN (…), EXISTS (…), corr[illegible]	[illegible] with EXPLAIN
4	Build `/dashboard` page: total students, total co[illegible]	[illegible] enrolment count.
5	Server-side pagination for `/students` with CO[illegible]	[illegible] query. Total count + page links.

SQL concepts introduced

- Aggregates: COUNT, SUM, AVG, MIN, MAX; COUNT(*) vs COUNT(col).
- GROUP BY and the rule that every non-aggregated select column must be in the GROUP BY.
- HAVING vs WHERE — which runs before/after aggregation.
- Subquery styles: scalar, IN, EXISTS, correlated, derived tables.
- FILTER clause for conditional aggregates (cleaner than CASE-inside-SUM).
- Pagination patterns: OFFSET/LIMIT vs keyset pagination.

Next.js concepts introduced

- URL search params as the source of truth for filters and page state.
- `<Link>` prefetching and pagination UX.
- Caching: `force-dynamic` vs default ISR for dashboards.

Conditional aggregate using FILTER

```
SELECT
  c.title,
  COUNT(*) AS total_enrolments,
  COUNT(*) FILTER (WHERE g.grade >= 70) AS passes,
  AVG(g.grade) AS avg_grade
FROM courses c
JOIN enrollments e ON e.course_id = c.id
LEFT JOIN grades g ON g.enrollment_id = e.id
GROUP BY c.id
HAVING COUNT(*) >= 3
ORDER BY avg_grade DESC NULLS LAST;
```

Practice exercises

- Find courses where the average grade is below 50 AND at least 5 students are enrolled.
- Find students whose average grade is in the top 10% (use a subquery on AVG).

- Use EXISTS to find students who have at least one failing grade.
- Write the dashboard 'top 5 courses by enrolment' two ways: subquery and JOIN.
- Add a date filter `?from=YYYY-MM-DD&to;=YYYY-MM-DD` to the dashboard and pass through to the SQL safely.

Definition of done for this week

- `grades` table exists with realistic data spanning multiple semesters.
- `/dashboard` shows at least 4 metrics from real aggregate queries.
- Pagination works on `/students` with correct total count.
- You can explain — out loud — why HAVING and WHERE run at different times.

Week 5 — Advanced SQL — CTEs, window functions, indexes, EXPLAIN

The advanced-SQL weekend. CTEs make complex queries readable; window functions unlock 'top N per group' and running totals; EXPLAIN ANALYZE makes you a real engineer who can defend a query plan. You will also add indexes and watch query time drop.

Hourly breakdown (5 hours)

Hour	Focus	Concrete output
1	CTEs (WITH …): readability, chained CTEs, materialised vs not. Recursive CTEs — one worked example (org chart or cat	Rewrite 2 nested subqueries as CTEs.
2	Window functions: OVER, PARTITION BY, ORDER BY, ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, SU	5 window-function queries written.
3	Indexes: B-tree, partial, multi-column, covering. When NOT to add an index. Watching pg_stat_user_indexes.	Add 3 indexes, verify usage with EXPLAIN.
4	EXPLAIN vs EXPLAIN ANALYZE. Reading the plan: seq scan vs index scan, nested loop vs hash join, rows estimated vs	Three slow queries diagnosed and fixed.
5	Build 'top 3 students per course by grade' page using a window function. Build 'grade trend per student' (LAG to compare	Two advanced screens live.

SQL concepts introduced

- Common Table Expressions (CTEs) and the WITH clause.
- Recursive CTEs for hierarchical / graph-ish data.
- Window functions and the OVER (PARTITION BY … ORDER BY …) syntax.
- Frame clauses (ROWS BETWEEN … and RANGE BETWEEN …).
- Index types in Postgres (B-tree default; GIN/BRIN exist for special cases).
- Reading EXPLAIN ANALYZE: rows, cost, actual time, loops.

Next.js concepts introduced

- Server-side data composition — multiple queries stitched in one Server Component.
- Streaming UI with Suspense boundaries for slow analytics queries.

Top 3 students per course (window function)

```
WITH ranked AS (
  SELECT
    s.name, c.title, g.grade,
    ROW_NUMBER() OVER (
      PARTITION BY c.id ORDER BY g.grade DESC
    ) AS rn
  FROM grades g
  JOIN enrollments e ON e.id = g.enrollment_id
  JOIN students s ON s.id = e.student_id
  JOIN courses c ON c.id = e.course_id
)
SELECT * FROM ranked WHERE rn <= 3;
```

Practice exercises

- Use a recursive CTE to build a category tree (add a `categories` table with a `parent_id`).
- Use LAG to compute each student's grade change between semesters.

- Use NTILE(4) to label students into quartiles by average grade.
- Add a partial index: `CREATE INDEX … ON students(email) WHERE deleted_at IS NULL;` and verify it is used.
- Take one slow page (>200ms), capture EXPLAIN ANALYZE before/after adding an index, paste both into your notes.

Definition of done for this week

- At least one CTE and at least one window function in production code.
- Three useful indexes added with reasoning written in a comment.
- Notes file contains at least one before/after EXPLAIN with timing.
- You can describe — without notes — what PARTITION BY does.

Indexing reality check: add an index only when you have a slow query and EXPLAIN shows a sequential scan on a big table. Indexes cost write performance and disk. 'Just add an index' is a smell.

Week 6 — Transactions, Prisma, auth, deploy — ship it

Tie everything together. Learn transactions properly, introduce Prisma so you feel what an ORM gives and takes away, add a thin auth layer so the app isn't wide open, and deploy to Vercel + Neon. End the plan with a real URL you can share.

Hourly breakdown (5 hours)

Hour	Focus	Concrete output
1	Transactions: BEGIN / COMMIT / ROLLBACK. [illegible] Isolation levels (briefly): READ COM	[illegible]
2	Install Prisma. Run `prisma db pull` to introspect your existing schema. Generate the client. Compare a Prisma query vs t	Prisma client generated; one query migrated.
3	Migrate the `students` CRUD module from raw `pg` to Prisma. Keep one screen on raw SQL on purpose — write a note a	Mixed-mode app working.
4	Add basic auth: `users` table, sign-in with email + password (bcrypt hash), iron-session or NextAuth Credentials. Protect r	Login screen + protected dashboard.
5	Push to GitHub. Deploy to Vercel. Move DB to Neon (or keep local + a tunnel). Smoke test the live URL.	Live app reachable; final commit pushed.

SQL concepts introduced

- Transactions and ACID — what each letter actually guarantees.
- Isolation levels and the anomalies each prevents (dirty read, non-repeatable read, phantom read, serialisation anomaly).
- SELECT … FOR UPDATE and row-level locking (one paragraph is enough at this stage).

Next.js concepts introduced

- Splitting your data layer: when Prisma helps, when raw SQL is still better (reporting, window-function-heavy queries).
- Auth basics with hashed passwords; never store plain-text.
- Environment management across local / preview / production on Vercel.
- Database connection limits on serverless (use Prisma Data Proxy or PgBouncer pooling).

Transactional enrol-and-grade using node-postgres

```
const client = await pool.connect();
try {
  await client.query('BEGIN');
  const e = await client.query(
    'INSERT INTO enrollments(student_id, course_id, semester) VALUES ($1,$2,$3)
RETURNING id',
    [studentId, courseId, semester]
  );
  await client.query(
    'INSERT INTO grades(enrollment_id, grade) VALUES ($1, NULL)',
    [e.rows[0].id]
  );
  await client.query('COMMIT');
} catch (err) {
  await client.query('ROLLBACK');
  throw err;
} finally {
  client.release();
}
```

Practice exercises

- Add a 'transfer enrolment from course A to course B' action implemented as a transaction.
- Deliberately throw mid-transaction; verify nothing was written.
- Migrate the courses CRUD to Prisma; compare lines of code and readability with the raw-SQL version.
- Add role to users (`admin` | `viewer`) and hide mutation buttons for viewers.
- Write a one-page `LEARNED.md` summarising the 5 most surprising things you picked up.

Definition of done for this week

- At least one multi-statement flow is wrapped in a transaction with rollback tested.
- Prisma is installed and at least one CRUD module uses it.
- App requires login for mutations; passwords are hashed.
- App is deployed to a public URL backed by managed Postgres.
- Git history tells the story: clear commits per week.

You did it. Six weekends, one Student Management System, and SQL that actually sticks. Keep the repo open, keep poking at queries, and bookmark the resources page — the difference between comfortable and confident is another 20 hours of deliberate practice on real problems.

Resources

Free, high-signal references to keep beside you:

- **PostgreSQL docs** — postgresql.org/docs (the single best SQL reference).
- **Use The Index, Luke** — use-the-index-luke.com (indexes & query plans).
- **PostgreSQL Tutorial** — postgresqltutorial.com (worked examples).
- **Next.js docs — Data Fetching & Server Actions** — nextjs.org/docs.
- **node-postgres (pg) docs** — node-postgres.com.
- **Prisma docs** — prisma.io/docs (introduce only in week 6).
- **SQLBolt** — sqlbolt.com (10-minute interactive drills).
- **pgexercises.com** — 80+ graded SQL exercises against a real schema.

After week 6 — where to go next

- Add full-text search (Postgres `tsvector` / `tsquery`).
- Add a reporting page using materialized views and refresh strategies.
- Introduce database migrations properly (Prisma Migrate or Drizzle Kit).
- Add tests: unit (vitest), integration (testcontainers-postgres), e2e (Playwright).
- Learn database backup/restore, role-based access, and row-level security.
- Read 'SQL Performance Explained' by Markus Winand — short, transformative.

Generated for gunapathi.selvam@accenture.com. Adjust pace freely — the plan is a scaffold, not a contract.